

ESTUDIO — Referencia rápida del JavaScript del frontend

Material de estudio para el final del TFC Bajoneá. **Referencia superficial, no exhaustiva** — pensada para tener a mano por si preguntan algo puntual de JS, no para estudiarla de memoria. Basado en `frontend/js/` — **16 archivos**, ~8500 líneas.

Lo primero, para ubicarte

El frontend de Bajoneá es **HTML, CSS y JavaScript puro** — **sin ningún framework**. No hay React, ni Angular, ni Vue.

Si te preguntan por qué: porque el alcance del proyecto no lo justificaba. Un framework agrega una capa de compilación, dependencias y complejidad que para 40 pantallas con formularios y listados no aporta nada. Con JS moderno (módulos ES6, `fetch`, `async/await`) alcanza de sobra.

Los tres conceptos que hay que tener claros

1. Módulos ES6. Cada archivo `.js` es un módulo: exporta funciones con `export` y usa las de otros con `import`.

```
import { apiFetch } from './api.js';
export async function initCatalogo() { ... }
```

2. `async` / `await`. Llamar al backend tarda. `async/await` permite escribir código que espera una respuesta **sin bloquear el navegador**:

```
const productos = await apiFetch('/catalogo/productos'); // espera acá, sin congelar la página
```

3. El patrón `initxxx()`. Cada pantalla tiene su función de inicialización, que el HTML importa y llama. Ejemplo: `catalogo.html` llama a `initCatalogo()`. Ese es el punto de entrada de cada pantalla.

Regla del proyecto que conviene saber

Ningún archivo de `frontend/` tiene comentarios. Ni de encabezado, ni explicativos, ni código comentado. Es una regla explícita del proyecto (CLAUDE.md, regla 11), verificada con `grep` sobre toda la carpeta. Los nombres de función son lo que documenta.

LOS 16 ARCHIVOS, DE UN VISTAZO

Archivo	Líneas	De qué se ocupa
<code>api.js</code>	148	El más importante. Toda comunicación con el backend pasa por acá.
<code>validators.js</code>	286	Las validaciones de formulario, espejo de las del backend.
<code>auth.js</code>	1655	Login, los dos registros, verificación, recuperación.
<code>catalogo.js</code>	1179	Catálogo, detalle de comercio, y los componentes compartidos de UI .
<code>comercio.js</code>	1820	El más grande. Todas las pantallas del rol comercio.
<code>admin.js</code>	1272	Todas las pantallas del administrador.
<code>checkout.js</code>	381	El checkout del pedido.
<code>cliente.js</code>	350	Perfil del cliente.
<code>pedidos.js</code>	339	Historial y detalle de pedidos del cliente.
<code>carrito.js</code>	295	La pantalla del carrito.
<code>explorar.js</code>	213	Búsqueda global de productos con filtros.
<code>crop.js</code>	162	El editor de recorte de imágenes.
<code>notificaciones.js</code>	145	La campanita y el listado, con polling.
<code>otp.js</code>	122	El input de código de 6 dígitos.
<code>cloudinary.js</code>	106	Subida de imágenes en dos pasos.
<code>geografia.js</code>	47	El selector Provincia → Localidad.

1. `api.js` — el archivo clave

Si te preguntan una sola cosa de JavaScript, va a ser sobre este archivo. Es el que centraliza absolutamente toda la comunicación con el backend.

Gestión de la sesión

Función	Qué hace
<code>getToken()</code>	Devuelve el JWT guardado en <code>localStorage</code> .
<code>setSesion(token, usuario)</code>	Guarda el token y los datos del usuario tras el login.
<code>getUsuario()</code>	Devuelve el usuario guardado (parseado del JSON).
<code>clearSesion()</code>	Borra token y usuario — se usa en el logout.

Se guarda en `localStorage`, que es el almacenamiento del navegador que sobrevive al cierre de la pestaña. Por eso seguís logueado cuando volvés al sitio.

(Si te preguntan por seguridad: `localStorage` es accesible desde JavaScript, así que un ataque de XSS podría leer el token. La alternativa más segura son cookies `HttpOnly`, pero requieren manejo de CSRF y una configuración más compleja. Es un trade-off conocido.)

apiFetch(path, opciones) — la función más importante de todo el frontend

Toda llamada al backend, sin excepción, pasa por acá. Hace seis cosas:

- Arma los headers, incluido `Authorization: Bearer <token>` si estás logueado.
- Aplica un timeout de 15 segundos con `AbortController` — si el backend no responde, corta.
- Hace el `fetch` y parsea la respuesta JSON.
- Maneja los errores globalmente (ver tabla abajo).
- Devuelve directamente el `data` del `ApiResponse`, no el objeto completo.
- Si algo falla de forma no manejable, lanza un `ApiError` con el status y los datos.

El manejo global de errores — lo mejor de este archivo

```
if (response.status >= 500)      → redirige a errores/500.html
if (response.status === 401 && teníasToken) → borra sesión + modal "Sesión cerrada"
if (response.status === 403 && teníasToken) → redirige a errores/acceso-denegado.html
if (error de red)                → redirige a errores/sin-conexion.html
if (timeout)                    → redirige a errores/timeout.html
```

Por qué esto está buenísimo: ninguna pantalla tiene que preocuparse por estos casos. Están resueltos en un solo lugar. Si mañana querés cambiar cómo se muestra un error de servidor, tocás `api.js` y listo — no 40 archivos.

El modal de "Sesión cerrada" — un detalle de UX que sale del backend

```
function showSesionCerradaModal() {
  crearModalSesionCerrada({
    titulo: 'Sesión cerrada',
    texto: 'Detectamos que iniciaste sesión en otro dispositivo. Por seguridad, tu sesión en este dispositivo fue cerrada automáticamente.',
  });
}
```

Esto es la contracara visible de la regla de sesión única del backend. Cuando el `JwtAuthenticationFilter` rechaza un token porque su sesión está `activa = false`, el frontend recibe un 401 y explica por qué, en vez de tirar al usuario al login sin decir nada.

También existe `mostrarModalCuentaBloqueada()` para el caso de los 3 intentos fallidos.

handle401Globally — el parámetro que evita un bug

```
apiFetch(path, { handle401Globally: false })
```

Sirve para desactivar el manejo automático del 401. **Hace falta en el login:** cuando escribís mal la contraseña, el backend devuelve 401, y sin este flag el frontend mostraría el modal de "sesión cerrada"... cuando en realidad nunca hubo sesión. Con el flag, esa pantalla maneja el 401 a su manera.

2. validators.js — las validaciones del lado del navegador

El concepto clave: son un espejo, no un reemplazo

Las validaciones del frontend NO reemplazan a las del backend, las duplican a propósito.

	Frontend	Backend
Para qué	Experiencia de usuario — feedback inmediato, sin esperar al servidor	Seguridad — es la validación real
¿Se puede saltar?	Sí, con las herramientas de desarrollo del navegador	No

Si te preguntan por qué están duplicadas, esa tabla es la respuesta. Un atacante puede desactivar el JavaScript o mandar el request directo con curl. El backend nunca confía en el frontend.

Las validaciones de formato (espejo de las anotaciones custom del backend)

Función	Qué valida	Equivale en el backend a
esEmailValido(email)	Formato de email	@ValidarFormatoEmail
esPasswordSegura(password)	8-72, mayúscula, minúscula, número	@ValidarPasswordSegura
esCuitValido(cuit)	11 dígitos + dígito verificador módulo 11	@ValidarCuit
esDniValido(dni) / esDniClienteValido(dni)	7 u 8 dígitos	@ValidarFormatoDni
esTelefonoValido(telefono)	+549 + 10 dígitos	@ValidarTelefonoArgentino
esNombrePropioValido(valor) / esNombreClienteValido(valor)	Solo letras, espacios, guiones	@ValidarFormatoNombre
esCodigoPostalValido(cp)	4 dígitos o CPA	@ValidarCodigoPostalArgentino
esFechaNacimientoValida(fecha)	Mayor de 18	@MayorDeEdad
esFechaNacimientoClientePlausible(fecha)	No futura, no más de 120 años	@ValidarFechaNacimientoPlausible
esFechaNoFuturaValida(fecha)	No puede ser futura	@PastOrPresent
esPrecioValido(valor)	Precio positivo y con formato	@Positive + @Digits
esUrlRedSocialValida(url)	Formato de link	El @Pattern de RedSocialRequestDTO
excedeSubtotalMaximo(precio, cantidad)	El tope de 99.999.999	La validación de PedidoService

El detalle que vale la pena mencionar: esCuitValido implementa el mismo algoritmo módulo 11 de AFIP que CuitValidator.java , incluida la constante CUIT_MULTIPLICADORES = [5,4,3,2,7,6,5,4,3,2] . El usuario se entera de que el CUIT está mal mientras lo escribe, sin esperar el viaje al servidor.

Las funciones de normalización (espejo de TextoUtils.java)

Función	Qué hace	Equivale a
aTitleCase(valor)	"JUAN carlos" → "Juan Carlos"	TextoUtils.aTitleCase
normalizarCodigoPostal(cp)	CPA a mayúsculas	TextoUtils.normalizarCodigoPostal
normalizarUrlRedSocial(url)	Le agrega https:// si falta	TextoUtils.normalizarUrlConEsquema
sanitizarDni(dni)	Saca puntos y guiones	El setter manual del DTO
sanitizarCuit(cuit)	Deja solo dígitos	El setter manual del DTO
colapsarEspacios(valor)	"Juan Carlos" → "Juan Carlos"	El setter manual del DTO

Sobre aTitleCase : es la misma lógica, portada a mano de Java a JavaScript, para que el usuario vea en el formulario exactamente lo que se va a guardar. Cuando se agregó en el backend, rompió 7 tests de Playwright que esperaban el texto sin normalizar — es un ejemplo real de por qué la sincronización importa.

Las funciones de UI de errores

Función	Qué hace
mostrarErrorCampo(errorElId, mensaje)	Pinta el mensaje de error debajo del input.
limpiarErrorCampo(errorElId)	Lo borra.
limpiarErroresCampos(ids)	Borra varios de una.
mapearErroresBackend(errores, mapa)	Toma el mapa {campo: mensaje} del backend y lo pinta en el input correcto.
validarCamposSilencioso(campos)	Valida sin mostrar errores — para habilitar/deshabilitar el botón.
validarCamposRequeridosSilencioso(campos)	Ídem, solo obligatorios.
scrollAlPrimerError(contenedor)	Lleva la vista al primer campo con error.
calcularFortalezaPassword(password)	Devuelve un nivel de fortaleza.
aplicarFortalezaPassword(password, barras, label)	Pinta la barrita de "débil / media / fuerte".

mapearErroresBackend es la contracara del data del GlobalExceptionHandler . El backend manda {"email": "Ingresá un email válido", "password": "..."} y esta función lo pinta debajo de cada input. Por eso el handler devuelve un mapa y no solo un string.

3. catalogo.js — el catálogo Y los componentes compartidos

Este archivo tiene doble función: las pantallas del catálogo y la biblioteca de componentes visuales que usa todo el frontend.

Los componentes compartidos (los usa medio proyecto)

Función	Qué hace
<code>renderTopBar(container, opciones)</code>	Dibuja el header: logo, volver, título, perfil, campanita, y una acción opcional.
<code>renderBottomNav(container, activo)</code>	La barra de navegación de abajo del cliente.
<code>showToast(mensaje, kind)</code>	Muestra un mensajito flotante ("Producto agregado").
<code>renderEmptyState(container, titulo, texto, opciones)</code>	El estado vacío ("No hay productos todavía").
<code>renderComercioCard(comercio)</code>	Dibuja la tarjeta de un comercio en el listado.
<code>pintarAvatarComercio(container, comercio)</code>	La foto redonda del comercio, con fallback si no tiene.
<code>pintarAvatarUsuario(container, url, texto)</code>	Ídem para el usuario.
<code>pintarEstadoComercio(el, abierto)</code>	El puntito verde/gris de "Abierto"/"Cerrado".
<code>renderPedidoEstadoHeader(container, opciones)</code>	El header de estado del pedido, compartido entre cliente y comercio .
<code>actualizarContadorCarrito()</code>	Actualiza el numerito del carrito en el header.
<code>crearAccionCarritoHeader()</code>	Arma el botón de carrito del header.
<code>manejarBloqueoPorCambioPassword(error)</code>	Maneja el caso especial de cuenta bloqueada.

Por qué están todos acá y no repetidos en cada pantalla: un solo cambio actualiza todas las pantallas a la vez. Cuando se rediseñaron los chips de estado de pedido, se tocó `renderPedidoEstadoHeader` una vez y quedaron corregidas las pantallas de cliente y de comercio.

estadoHorario(horarios) — el espejo de estaAbiertoAhora del backend

Calcula, del lado del navegador, si el comercio está abierto ahora mismo, mirando los horarios que vinieron en el DTO. Es la misma lógica que `ComercioService.estaAbiertoAhora` , duplicada para poder pintar el estado sin una llamada extra.

Las pantallas

Función	Pantalla	Qué hace
<code>initCatalogo()</code>	<code>catalogo.html</code> / <code>index.html</code>	Trae los comercios aprobados, los ordena (abiertos primero) y los pinta con chips de filtro.
<code>initComercioDetalle()</code>	<code>comercio-detalle.html</code>	El perfil del comercio con sus productos, el modal de producto y el stepper de "agregar al carrito".

4. auth.js — el archivo de autenticación (1655 líneas)

Función	Pantalla / uso	Qué hace
<code>initLogin()</code>	<code>login.html</code>	Valida, llama a <code>POST /auth/login</code> , guarda la sesión y redirige según el rol.
<code>resolverHomePorRol(usuario)</code>	—	Decide a dónde mandar a cada rol tras el login.
<code>initRegistroCliente()</code>	<code>registro-cliente.html</code>	El wizard de registro de cliente, paso por paso.
<code>initRegistroComercio()</code>	<code>registro-comercio.html</code>	El wizard más complejo: datos legales, comercio, horarios, redes sociales, representante, foto.
<code>initVerificarEmail()</code>	<code>verificar-email.html</code>	El input de 6 dígitos + reenvío de código.
<code>initRecuperarPasswordSolicitar()</code>	<code>recuperar-password.html</code>	Los 3 pasos: pedir código, validarlo, cambiar la contraseña.
<code>initReactivarCuentaSolicitar()</code>	<code>reactivar-cuenta.html</code>	Los 2 pasos de reactivación.
<code>logout(destino)</code>	Todas	Llama a <code>POST /auth/logout</code> , borra la sesión local y redirige.
<code>construirTelefono(digitos)</code>	—	Concatena el prefijo +549 antes de mandar.

resolverHomePorRol(usuario) — el ruteo por rol

Después del login, cada rol va a un lugar distinto:

- **CLIENTE** → el catálogo.
- **ADMINISTRADOR** → `admin-dashboard.html` .
- **DUENO** → depende del **estado de su comercio**: si está aprobado, al dashboard; si está pendiente o rechazado, a las pantallas correspondientes.

Es un guard de negocio, no solo de rol. `SecurityConfig` protege por rol, pero **no** por estado del comercio — eso lo resuelve el frontend.

construirTelefono(digitos) — el par de @ValidarTelefonoArgentino

El campo del prefijo `+549` en el HTML **no es editable**; esta función lo concatena antes de enviar. **Por eso el backend puede rechazar un teléfono sin prefijo en vez de completarlo**: desde la app propia siempre viene bien, así que un valor sin prefijo solo puede venir de un cliente de API fuera de contrato.

Las constantes de etiquetas

```
export const LABELS_TIPO_SOCIEDAD = { SA: 'Sociedad Anónima', SRL: '...' }
export const LABELS_CONDICION_IVA = { ... }
export const LABELS_TIPO_COMERCIO = { ... }
export const LABELS_TIPO_RED_SOCIAL = { ... }
```

Traducen los valores técnicos de los enums a texto legible. El backend manda `RESPONSABLE_INSCRIPTO` ; el usuario lee "Responsable Inscripto".

(Es la misma idea que `MotivoRechazo.getEtiqueta()` del backend, resuelta acá del lado del cliente para los enums que no tienen etiqueta propia.)

5. comercio.js — el archivo más grande (1820 líneas)

Función	Pantalla	Qué hace
<code>initComercioEstadoPagina(estadoEsperado)</code>	Todas	Guard de estado : verifica que el comercio esté en el estado que esa pantalla espera.
<code>initComercioDashboard()</code>	<code>comercio-dashboard.html</code>	Trae el resumen del día y los pedidos activos.
<code>initComercioPedidos()</code>	<code>comercio-pedidos.html</code>	Lista los pedidos con chips de filtro por estado.
<code>initComercioPedidoDetalle()</code>	<code>comercio-pedido-detalle.html</code>	El detalle, con aceptar/rechazar solo si está PENDIENTE .
<code>initComercioPerfil()</code>	<code>comercio-perfil.html</code>	Ver y editar el perfil, incluida la foto.
<code>initComercioProductos()</code>	<code>comercio-productos.html</code>	El listado de productos del comercio.
<code>initComercioProductoForm()</code>	<code>comercio-producto-form.html</code>	Alta y edición de producto, con la galería.
<code>renderBottomNavComercio(container, activo)</code>	—	La barra de navegación del comercio.

initComercioEstadoPagina(estadoEsperado) — el guard que el backend no hace

Por qué existe: `SecurityConfig` protege las rutas **por rol**, pero un dueño con comercio `PENDIENTE` tiene el mismo rol `DUENO` que uno aprobado. **El backend no distingue**.

Esta función consulta el perfil del comercio y, si el estado no es el que la pantalla espera, redirige (a "pendiente de aprobación" o a "rechazado").

Ojo con cómo lo contás: *"es un guard de experiencia de usuario, no de seguridad"*. La seguridad real está en el backend: aunque alguien saltee esta pantalla a mano, `validarAceptaPedidos` del `ComercioService` no lo va a dejar operar.

Un detalle de diseño de initComercioPedidoDetalle

Cuando el pedido está `EN_PREPARACION` , la pantalla no muestra ningún botón de acción.

Es a propósito: `EstadoPedido` no tiene ninguna transición implementada después de `EN_PREPARACION` (no existe "marcar como entregado"). **Mostrar un botón que no hace nada sería peor que no mostrarlo**. Es el mismo criterio con el que se omitió la campanita del header de administrador — no hay ninguna notificación que apunte a un admin, así que un ícono siempre en cero sería un control sin función.

6. admin.js — el panel de administrador (1272 líneas)

Función	Pantalla	Qué hace
<code>initAdminDashboard()</code>	<code>admin-dashboard.html</code>	Las métricas y los accesos rápidos.
<code>initAdminComerciosPendientes()</code>	<code>admin-comercios-pendientes.html</code>	La cola de aprobación.
<code>initAdminComercioDetalle()</code>	<code>admin-comercio-detalle.html</code>	La ficha completa + modales de aprobar/rechazar.
<code>initAdminComercios()</code>	<code>admin-comercios.html</code>	Los comercios aprobados, con modal de detalle.
<code>initAdminClientes()</code>	<code>admin-clientes.html</code>	El listado de clientes (solo lectura).
<code>initAdminCategorias()</code>	<code>admin-categorias.html</code>	CRUD de categorías.
<code>initAdminTags()</code>	<code>admin-tags.html</code>	CRUD de tags.

El flujo de rechazo pide el motivo en un modal, con el campo obligatorio — el espejo de la validación de `AdministradorService.resolverAprobacion` .

7. carrito.js , checkout.js , pedidos.js , cliente.js , explorar.js

Archivo	Función principal	Qué hace
carrito.js	initCarrito()	Muestra el carrito con foto de cada producto, permite cambiar cantidades, eliminar, y vaciar.
checkout.js	initCheckout()	Elegir delivery o retiro, mostrar la dirección, confirmar el pedido.
pedidos.js	initPedidosHistorial()	El historial de pedidos del cliente.
pedidos.js	initPedidoDetalle()	El detalle de un pedido puntual.
cliente.js	initPerfil()	Ver y editar el perfil, con foto y cambio de contraseña.
explorar.js	initExplorar()	Búsqueda global de productos con filtros por categoría y tags.

Dos detalles de implementación que pueden preguntarte

1. `initPedidoDetalle` no llama a un endpoint de detalle. No existe `GET /pedidos/{id}` , así que llama a `GET /pedidos/cliente` (que ya viene filtrado por el JWT) y **busca el pedido en la lista**.

El beneficio: como el listado ya está filtrado por el backend, **es imposible que te devuelva un pedido ajeno**. Un endpoint por id tendría que verificar el dueño a mano.

2. `carrito.js` tiene un `try/catch` que corrige un bug real. Cuando se agregó la foto del producto a cada línea del carrito, había que traerla con `GET /catalogo/comercios/{id}/productos` . Pero si tenías un carrito viejo de un comercio que ya **no está aprobado**, ese endpoint devuelve 404 y **la pantalla entera quedaba en blanco**. Se resolvió con un `try/catch` que degrada a "sin foto" en vez de romper.

8. notificaciones.js — el polling

Función	Qué hace
<code>initNotificaciones()</code>	Lista las notificaciones y permite marcarlas como leídas.

El polling — pregunta probable

El archivo tiene un `setInterval` que **cada 15 segundos** llama a `GET /notificaciones/no-leidas/contador` y actualiza el numerito de la campanita.

cada 15s → `GET /notificaciones/no-leidas/contador` → actualiza el badge

No es tiempo real de verdad. La alternativa serían **WebSockets** (el servidor te avisa cuando pasa algo), que es instantáneo pero mucho más complejo: requiere mantener una conexión abierta por usuario y manejar reconexiones.

Por qué polling alcanza acá: la escala es chica y 15 segundos de demora en enterarte de un pedido no rompe nada. Es la decisión de "lo simple que funciona" en vez de "lo sofisticado que no hacía falta".

Lo que **sí se optimizó**: el polling llama al endpoint del **contador**, que devuelve solo un número. La lista completa se trae únicamente cuando el usuario abre la campanita.

9. cloudinary.js — la subida de imágenes

Función	Qué hace
<code>validarArchivoImagen(file)</code>	Antes de subir : valida tipo (jpg/png/webp) y tamaño (máx. 5 MB).
<code>subirImagenProducto(productoId, file, opciones)</code>	El flujo completo de 2 pasos para una imagen de producto.
<code>subirFotoPerfilComercio(file)</code>	Ídem para la foto del comercio.
<code>subirFotoPerfilUsuario(usuarioId, file)</code>	Ídem para la foto del usuario.
<code>subirFotoPerfilRegistroComercio(file)</code>	Durante el registro (sin login).
<code>subirFotoPerfilRegistroCliente(file)</code>	Ídem para cliente.
<code>eliminarImagenProducto(productoId, imagenId)</code>	Borra una imagen.
<code>reordenarImagenProducto(productoId, imagenId, orden)</code>	Cambia el orden.
<code>recortarImagenProducto(productoId, imagenId, file)</code>	Sube la versión recortada.
<code>eliminarFotoPerfilUsuario(usuarioId)</code>	Saca la foto de perfil.

subirArchivoConFirma(firma, file) — el corazón del flujo

```
const formData = new FormData();
formData.append('file', file);
formData.append('api_key', firma.apiKey);
formData.append('timestamp', String(firma.timestamp));
formData.append('signature', firma.signature);
formData.append('folder', firma.folder);
formData.append('upload_preset', firma.uploadPreset);

await fetch(`https://api.cloudinary.com/v1_1/${firma.cloudName}/image/upload`, {
  method: 'POST', body: formData
});
```

Fijate a dónde apunta el `fetch` : a `api.cloudinary.com` , NO al backend de Bajoneá.

Eso es la prueba concreta de que el archivo nunca pasa por el servidor propio. El flujo es:

1. Frontend → Backend: "dame una firma" (POST ../firma)
2. Backend → Frontend: la firma + metadatos
3. Frontend → Cloudinary: el archivo + la firma (POST directo)
4. Cloudinary→ Frontend: la URL de la imagen
5. Frontend → Backend: "guardá esta URL" (POST/PUT/PATCH)

Ventajas: el servidor no gasta ancho de banda ni disco, y la firma —atada a una carpeta específica— garantiza que la imagen se guarde donde corresponde.

Validación duplicada, otra vez

`validarArchivoImagen` chequea tipo y tamaño antes de subir. Es UX: le avisás al usuario al instante en vez de esperar a que suba 8 MB para que Cloudinary lo rechace.

Pero la validación real está en el **Upload Preset de Cloudinary**, configurado del lado del servicio. El frontend valida por cortesía; **Cloudinary valida de verdad**.

10. geografia.js — el selector dependiente

Función	Qué hace
<code>initGeografiaSelects(provinciaSelect, localidadSelect, provinciaPreferida)</code>	Arma el par de selectores.
<code>cargarLocalidades(provinciaId, localidadSelect)</code> (interna)	Trae las localidades de una provincia.

Cómo funciona el selector dependiente:

1. Al arrancar, el selector de localidad está **deshabilitado**, con el texto "Elegí una provincia primero".
2. Se cargan las 24 provincias con `GET /geografia/provincias` .
3. Al cambiar la provincia, se dispara `cargarLocalidades` , que muestra "Cargando localidades..." y trae `GET /geografia/localidades?provinciaId=X` .
4. Recién ahí se habilita el selector de localidad.

El **parámetro** `provinciaPreferida` permite preseleccionar Tierra del Fuego, que es donde opera Bajoneá. Un detalle chico de usabilidad: la mayoría de los usuarios no tiene que buscar su provincia.

11. otp.js — el input de código de 6 dígitos

Función	Qué devuelve
<code>crearInputOtp(container, { onComplete })</code>	Un objeto con métodos para controlar el input.

Los métodos del objeto que devuelve: `getValor()` , `focus()` , `reset()` , `marcarError()` , `marcarExito()` , `setDisabled(bool)` .

Los detalles de usabilidad que tiene (más de los que parece)

- **Avanza solo:** escribís un dígito y el foco salta a la casilla siguiente.
- **Retrocede con Backspace:** si la casilla está vacía y apretás borrar, vuelve a la anterior.
- **Pegar funciona:** si copiás el código de 6 dígitos del mail y lo pegás, se reparte entre las 6 casillas solo.
- **Solo números:** `input.value.replace(/\D/g, '')` filtra cualquier letra en el momento.
- `autocomplete="one-time-code"` en la primera casilla: en un celular, el sistema operativo ofrece el código del SMS/mail automáticamente.
- `onComplete` dispara la validación sola al completar los 6 dígitos, sin apretar ningún botón.
- **Estados visuales:** `marcarError()` pinta las casillas de rojo, `marcarExito()` de verde.

Es un **componente reutilizado en 3 pantallas**: verificación de email, recuperación de contraseña y reactivación de cuenta. Es el correlato visual de los códigos de 6 dígitos que genera `AuthService.generarValorToken` .

12. crop.js — el editor de recorte

Función	Qué hace
<code>abrirEditorRecorte({ origen, aspectRatio, onConfirmar, onCancelar })</code>	Abre el modal para recortar una imagen antes de subirla.

Para qué sirve: las fotos de producto se muestran en formato 4:3. Si el usuario sube una foto vertical, se vería mal recortada automáticamente. Este editor le deja **elegir qué parte se ve**.

El `aspectRatio` **está fijado en 4:3**, y de ahí sale el texto de "resolución recomendada: 1200×900px" que aparece en el formulario de producto — no es un número inventado, se deriva del recorte real.

Usa los endpoints `/recorte/firma` y `/url` de `ProductoController` : sube la versión recortada como un archivo nuevo y reemplaza la URL de la imagen existente.

Preguntas típicas de mesa sobre el frontend

"¿Usaste algún framework?" No, HTML, CSS y JavaScript puro con módulos ES6. Para el alcance del proyecto —formularios y listados— un framework agregaba complejidad sin aportar nada.

"¿Cómo hacés las llamadas al backend?" Todo pasa por una función `apiFetch` en `api.js` , que arma el header con el JWT, aplica un timeout de 15 segundos, y maneja globalmente los errores 401, 403, 500, sin conexión y timeout. Ninguna pantalla repite ese código.

"¿Dónde guardás el token?" En `localStorage` , así sobrevive al cierre de la pestaña. La contrapartida es que es accesible desde JavaScript; una cookie `HttpOnly` sería más segura pero requiere manejar CSRF.

"¿Por qué validás en el frontend si ya validás en el backend?" Son propósitos distintos: el frontend valida para dar feedback inmediato al usuario, el backend valida por seguridad. Las del frontend se pueden saltar con las herramientas de desarrollo; el backend nunca confía en lo que le llega.

"¿Cómo funcionan las notificaciones?" Con polling: un `setInterval` que cada 15 segundos consulta el contador de no leídas. No es tiempo real; la alternativa serían WebSockets, más complejos de lo que este proyecto necesita.

"¿Cómo subís las imágenes?" En dos pasos: el frontend pide una firma al backend, sube el archivo **directo a Cloudinary** con esa firma, y después le manda al backend solo la URL. El `fetch` de subida apunta a `api.cloudinary.com` , no a mi servidor.

"¿Qué es el patrón `initxxx` ?" Cada pantalla exporta una función de inicialización que su HTML importa y llama. Es el punto de entrada de esa pantalla: trae los datos, pinta el DOM y engancha los eventos.

Índice de archivos cubiertos en este documento

Archivo	En una línea
<code>api.js</code>	<code>apiFetch</code> + gestión de sesión + manejo global de errores. El más importante.
<code>validators.js</code>	Las validaciones y normalizaciones, espejo de las del backend.
<code>auth.js</code>	Login, los dos registros, verificación, recuperación, logout.
<code>catalogo.js</code>	Catálogo, detalle de comercio, y los componentes visuales compartidos.
<code>comercio.js</code>	Todas las pantallas del rol comercio. El archivo más grande.
<code>admin.js</code>	Todas las pantallas del administrador.
<code>carrito.js</code>	La pantalla del carrito.
<code>checkout.js</code>	La confirmación del pedido.
<code>pedidos.js</code>	Historial y detalle de pedidos del cliente.
<code>cliente.js</code>	Perfil del cliente.
<code>explorar.js</code>	Búsqueda global con filtros.
<code>notificaciones.js</code>	La campanita y el listado, con polling cada 15 s.
<code>cloudinary.js</code>	La subida de imágenes en dos pasos.
<code>geografia.js</code>	El selector dependiente Provincia → Localidad.
<code>otp.js</code>	El input de código de 6 dígitos, reutilizado en 3 pantallas.
<code>crop.js</code>	El editor de recorte 4:3.